

DKU ePrint for Linux

A Linux/CUPS driver for Duke Kunshan's ePrint service, which runs **Pharos Uniprint**. Pharos ships clients for macOS and Windows only; this is a clean reimplementation of the bits Linux needs.

It provides:

- a `popup:// CUPS backend` that speaks the Pharos popup protocol and delivers jobs over LPD,
- the three vendor **PPDs** (Ricoh MP 5002, Ricoh MP C4503, Lexmark MS810), lifted unchanged from the official `MACePrint.dmg`,
- `dku-eprint`, a small CLI to configure and test it.

Jobs are held at the server and released with your DKUCard at any ePrint station, exactly as with the macOS client.

Install

One command, on Debian/Ubuntu, Fedora/RHEL or Arch:

```
git clone https://github.com/pomegranar/dku-eprint-linux
cd dku-eprint-linux && sudo ./install.sh
```

It installs any missing dependencies with your distro's package manager, sets up the backend and PPDs, creates the three queues (`ePrint-Ricoh-BW`, `ePrint-Ricoh-Color`, `ePrint-Lexmark-BW`, matching the macOS installer), and asks how you want to supply your NetID.

Non-interactive:

```
sudo ./install.sh --netid abc123          # store it, use it for every job
sudo ./install.sh --prompt always        # ask in a dialog, every job
```

Verify:

```
dku-eprint probe          # ask the server what each queue requires
dku-eprint test-page      # send a real test page
dku-eprint print file.pdf
```

Remove it with `sudo ./uninstall.sh` (add `--purge` to drop the config too).

Distro packages

```
make rpm      # Fedora/RHEL    -> build/rpm/RPMS/noarch/
make deb      # Debian/Ubuntu -> ../dku-eprint_1.0.0*.deb
make arch     # Arch          -> build/arch/
```

Only the RPM has actually been built and inspected. The `debian/` files and the `PKGBUILD` are written to spec and syntax-checked, but they have never been run: the machine this was developed on has no `dpkg-buildpackage` or `makepkg`. Treat `make deb`

and `make arch` as untested, and expect to fix something the first time you build them.

The same caveat applies more weakly to `install.sh`: it was developed and run on Fedora, so the `apt` and `pacman` branches of its dependency handling have not been exercised on a real machine either. Reports and patches welcome.

The packages are self-contained: installing one creates the three queues, enables the NetID dialog agent, and needs no terminal afterwards. Install it from your graphical software centre, print, type your NetID into the dialog that appears, and collect the job with your DKUCard.

If `cupsd` was not running at install time the queues cannot be created, and the package says so; finish with `sudo dku-eprint add-queues`.

Printing

```
dku-eprint print report.pdf                # default queue, ePrint-Ricoh-BW
dku-eprint print --queue ePrint-Ricoh-Color *.pdf # colour
dku-eprint print -n 2 --sides two-sided-long-edge --pages 1-4 report.pdf
dku-eprint print --netid xyz789 report.pdf    # charge someone else's account
cat report.pdf | dku-eprint print            # standard input
```

Anything `lp(1)` understands can be passed through with `-o`, and `dku-eprint print` is itself only a wrapper over `lp`, so this remains equivalent:

```
lp -d ePrint-Ricoh-BW report.pdf
```

Either way the file goes through the CUPS filter chain for the queue's PPD, so PDFs, images and plain text all print. Jobs are held at the server; release them with your DKUCard at any ePrint station.

Requirements

- CUPS and `python3` (standard library only, no third-party packages)
- Network reach to `dku-ep-ps2-pap1.oit.duke.edu` on TCP **515** and **28203**

VPNs and split tunnelling

The print server is on campus RFC 1918 space (`10.0.0.0/8`). If you run a VPN that captures the default route (a Tailscale exit node, WireGuard with `AllowedIPs = 0.0.0.0/0`, a corporate client), campus traffic gets tunnelled off site and the printer looks dead.

The only thing that matters is that the server resolves via your LAN interface:

```
ip route get "$(getent hosts dku-ep-ps2-pap1.oit.duke.edu | awk '{print $1}')"
# want: 'dev <your LAN interface>', not 'dev tailscale0' / 'dev wg0'
```

If it points at the tunnel, you don't need to disconnect. Just exempt the campus range. For Tailscale, its own rules sit at priorities 5210-5270, so a lower number wins:

```
sudo ip rule add to 10.0.0.0/8 lookup main priority 5206
```

Note that Tailscale's `--exit-node-allow-lan-access` alone is usually *not* enough: it adds a `throw` route for your immediate subnet only, and the print server generally lives on a different one.

`ip rule` changes are **runtime only**: lost on reboot and on VPN restart. To persist, use a NetworkManager dispatcher script (`/etc/NetworkManager/dispatcher.d/`) or a small systemd unit. If printing works today but breaks after a reboot, check this first:

```
ip rule show | grep 5206
```

Dropping the exit node entirely (`tailscale set --exit-node=`) also works, and is less disruptive than disconnecting from your tailnet.

Configuration

`/etc/dku-eprint/eprint.conf`:

```
server = dku-ep-ps2-pap1.oit.duke.edu
netid   = abc123
```

```
# never = always use the netid above
# always = ask in a desktop dialog for every job
prompt = always
```

```
# Optional per-account overrides for shared machines
[users]
alice = abc123
bob   = xyz789
```

```
sudo dku-eprint set-netid abc123
sudo dku-eprint set-netid xyz789 --user bob
dku-eprint show
```

Two ways to supply the NetID

`prompt = never` uses the stored NetID for every job. Simplest, and the right choice for headless and SSH printing. `install.sh` offers it as option 1.

`prompt = always` reproduces the macOS client: a dialog appears for each job. This is what the distro packages default to, since a package has no way to ask you for a NetID while it installs.

```
sudo dku-eprint set-prompt always
systemctl --user enable --now dku-eprint-agent.service # as your own user
```

A CUPS backend cannot open a window itself. It runs as root with no session, no display, and often before you are back at the machine. So the same split the macOS client uses applies here: `dku-eprint-agent` runs inside your desktop session, and the backend hands it the question over a socket in `$XDG_RUNTIME_DIR`. It shows the dialog with `zenity` or `kdiallog`.

If the agent is not running (headless, SSH, or a different user), the backend logs a warning and falls back to the stored NetID rather than failing the job. Cancelling the dialog cancels the print, quietly.

The backend also asks the server which questions the queue wants, so if DKU ever adds one, you get a clear log line rather than a silently malformed job.

How it works

The DKU queues are ordinary LPD queues that expect a Pharos “popup block” in front of the print data, carrying the username and the answers to the queue’s questions. A separate service on port 28203 exists only so clients can discover what those questions are.

For DKU, every queue asks exactly one: *“Please enter your DKU NetID”*.

The full wire format (packet framing, the 32-byte block header, the four body sections, and the RC4 obfuscation) is documented in `docs/PROTOCOL.md`, along with how it was recovered from the vendor binary.

Tests

```
python3 -m unittest discover -s tests
```

60 tests: wire-format checks against captured bytes, RC4 against a published vector, NetID validation, the agent socket protocol, the PPD patching described in `THIRD-PARTY.md` (validated with `cupstestppd` where it is installed), and a full backend run driven exactly as `cupsd` drives it against a stub LPD server. They need no network and touch nothing outside `/tmp`.

Layout

<code>src/eprint_pharos.py</code>	protocol library (popup protocol, block builder, LPD)
<code>src/eprint_config.py</code>	config file handling
<code>src/popup</code>	the CUPS backend
<code>src/dku-eprint</code>	CLI: printing, configuration, diagnostics
<code>src/dku-eprint-agent</code>	per-user dialog agent for prompt = always
<code>packaging/</code>	rpm spec, debian/, PKGBUILD, systemd user unit
<code>vendor/ppd/</code>	PPDs from MACePrint.dmg, unmodified
<code>tools/prepare-ppds.sh</code>	strips the macOS-only filter at install time
<code>docs/PROTOCOL.md</code>	wire format documentation
<code>tests/</code>	unit and end-to-end tests

LICENSE	MIT, for the original work
THIRD-PARTY.md	provenance of vendor PPDs and the RE basis

License

The original work here (`src/`, `tests/`, `docs/`, and the install scripts) is MIT licensed. See `LICENSE`.

The PPDs in `vendor/ppd/` are Ricoh's and Lexmark's, redistributed unmodified for interoperability and **not** covered by that license. Provenance and the reverse-engineering basis are set out in `THIRD-PARTY.md`.

Not affiliated with or endorsed by Pharos Systems International, Ricoh, Lexmark, Duke University, or Duke Kunshan University.

Scope and caveats

- Only the plaintext LPD path is implemented. The vendor backend can also wrap the stream in AES after fetching a public key from the LPD server; DKU's server accepts plaintext, which is the vendor client's own documented fallback.
- Cost-centre and guest-account question types are encoded per the vendor format but are untested. DKU's queues don't use them.